

What is a Thread?

A thread is the lowest-level unit of execution in your program. It represents an actual path of instructions that the CPU executes.

Think of threads as workers in a factory:

- Each worker can do one job at a time.
- The more workers you have, the more jobs you can do concurrently.
- But workers (threads) consume system resources — they're not cheap to create or maintain.

In .NET:

```
new Thread(() => DoWork()).Start();
```

This directly creates and starts a new thread — an independent worker executing `DoWork()`.

Key characteristics:

- Threads are managed by the OS or .NET runtime.
 - They are expensive to create and manage.
 - You control when they start and stop.
 - They're not inherently asynchronous — they just run code concurrently.
-

What is a Task?

A Task is a higher-level abstraction built *on top of threads*.

It represents a unit of work that may run asynchronously — but you don't care *how* it runs (or on which thread).

Think of a task as a job assignment — the system decides which worker (thread) will handle it.

In .NET:

```
await Task.Run(() => DoWork());
```

This runs `DoWork()` asynchronously, without you managing the thread directly.

Key characteristics:

- Tasks are part of the Task Parallel Library (TPL).
- They can run on thread pool threads, not new ones every time.
- They support continuations (what happens after they complete).
- They integrate with `async/await`, making async code readable and scalable.

Key Differences

Feature	Thread	Task
Level	Low-level (OS concept)	High-level (abstraction over threads)
Creation	<code>new Thread()</code>	<code>Task.Run()</code> or <code>Task.Factory.StartNew()</code>
Control	Manual start/stop/join	Managed by the Task Scheduler
Async Integration	No direct support	Works seamlessly with <code>async/await</code>
Resource Usage	Heavier (creates a new thread)	Lighter (uses thread pool)
Purpose	Executes code concurrently	Represents an asynchronous operation
Error Handling	Manual (<code>try/catch</code> inside thread)	Built-in exception propagation

Analogy

Imagine you run a delivery company:

- **Threads** = The drivers. You hire, train, and manage each one yourself.
- **Tasks** = The delivery jobs. You just submit a delivery order, and the system assigns an available driver automatically.

You focus on *what* needs to be done (tasks), not *who* does it (threads).